Convolutional Generative Adversarial Network for MNIST Digit Synthesis

R. Kumar
Department of Mechanical Engineering
University of Tennessee at Chattanooga, TN, 37403, USA

***Abstract*—This report presents a convolutional generative adversarial network (CNN-GAN) trained to synthesize handwritten digits from the MNIST dataset. Starting from the tutorial multilayer perceptron (MLP) GAN baseline, the generator and discriminator are replaced with convolutional architectures while preserving all other design choices: dataset normalization, optimizer configuration, and the adversarial training protocol. The CNN generator maps a 100-dimensional latent vector to a $1\times28\times28$ image through a projection, two spatial upsample-then-convolve blocks, and a Tanh output; the CNN discriminator applies three strided convolution stages before a single sigmoid output neuron. After 10 000 training epochs on an NVIDIA GPU, the model produces recognizable digit structure that visually surpasses the MLP baseline, validating the convolutional inductive bias for image synthesis.**

I. Introduction

A persistent limitation of feed-forward generative models is their inability to exploit the spatial structure of natural images. Fully connected layers treat every pixel as an independent feature and must learn translation invariance from scratch, which is both data-inefficient and capacity-intensive. The rise of convolutional neural networks (CNNs) for discriminative tasks motivated their adoption in generative settings, most notably in the generative adversarial network (GAN) framework introduced by Goodfellow et al. [1].

A GAN places two networks in adversarial competition: a generator G that produces synthetic data from random noise, and a discriminator D that attempts to distinguish real from generated samples. Neither network is given explicit supervision; instead, the signal flows entirely through the adversarial objective. This self-supervised dynamic allows GANs to produce samples of remarkable perceptual quality while requiring only unlabeled data.

The convolutional extension of GANs, formally described by Radford et al. as the Deep Convolutional GAN (DCGAN) [2], replaced pooling with strided convolutions in D and transposed convolutions (or upsample-then-convolve blocks) in G. This architectural shift substantially improved image quality and training stability.

This study replicates the DCGAN principles on MNIST [3] using a simplified upsample-convolution generator and a strided-convolution discriminator. Section II reviews GAN theory. Section III describes both the high-level GAN architecture and the specific CNN design. Section IV details the implementation. Sections V and VI present and interpret the results.

II. Theory

A. Adversarial Objective

Let $\mathbf{x} \sim p_{\text{data}}$ be a real image and $\mathbf{z} \sim \mathcal{N}(\mathbf{0}, \mathbf{I}_{100})$ be a latent noise vector. $G : \mathbb{R}^{100} \to \mathbb{R}^{C\times H\times W}$ maps noise to a synthetic image; $D : \mathbb{R}^{C\times H\times W} \to [0, 1]$ estimates the probability that an image is real. The minimax objective is:

$$\min_G \max_D \; \mathbb{E}_{\mathbf{x}}[\log D(\mathbf{x})] + \mathbb{E}_{\mathbf{z}}[\log(1 - D(G(\mathbf{z})))]. \quad (1)$$

In practice, the generator is trained to maximize $\log D(G(\mathbf{z}))$ rather than minimize $\log(1 - D(G(\mathbf{z})))$ to avoid vanishing gradients early in training [1].

B. Training Dynamics

D and G are updated in alternating steps. At each iteration, D minimizes the sum of two binary cross-entropy losses:

$$\mathcal{L}_D = -\tfrac{1}{2}\big[\log D(\mathbf{x}) + \log(1 - D(G(\mathbf{z})))\big]. \quad (2)$$

G then minimizes:

$$\mathcal{L}_G = -\log D(G(\mathbf{z}')), \quad (3)$$

where $\mathbf{z}'$ is a fresh noise sample drawn independently of $\mathbf{z}$. The theoretical fixed point occurs when G has recovered the data distribution and $D(\mathbf{x}) = \frac{1}{2}$ everywhere [1]; in practice, this equilibrium is rarely reached but is approximated sufficiently well to produce high-quality samples.

C. Role of Convolutional Architectures

Convolutional layers impose *weight sharing* and *local connectivity*, encoding the inductive bias that nearby pixels are more statistically dependent than distant ones. For the discriminator, strided convolutions progressively downsample feature maps, learning hierarchical texture and shape detectors without explicit pooling. For the generator, transposed convolutions or upsample-then-convolve blocks progressively increase spatial resolution from a compact latent code. Batch normalization [4] stabilizes training by reducing internal covariate shift, and is applied after each convolutional block in both networks (with the exception of the generator's output layer and the discriminator's input layer, following DCGAN guidelines [2]).

III. Architecture

A. Overall GAN Framework

Figure 1 shows the high-level GAN data flow. The generator receives a random latent vector $\mathbf{z}$ and produces a 28×28 grayscale image. The discriminator receives either a real MNIST image or a generator output and produces a scalar confidence score. Gradients from the discriminator's real/fake predictions flow back to update each network on alternating steps.

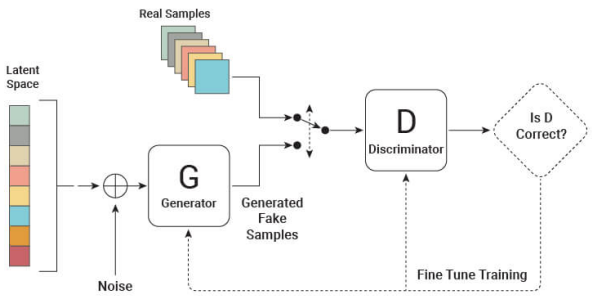

Fig. 1. GAN framework. The generator G maps latent noise $\mathbf{z}$ to a synthetic image; the discriminator D outputs a real/fake score for both real data $\mathbf{x}$ and generated samples $G(\mathbf{z})$. The two networks are updated by alternating gradient steps driven by the adversarial objective in (1).

B. CNN Generator and Discriminator

Figure 2 illustrates the convolutional architecture used in this work. The generator begins with a dense projection of $\mathbf{z}$ into a $128 \times 7 \times 7$ spatial feature map. Two upsample-then-convolve blocks each double the spatial resolution, yielding a $64{\times}28{\times}28$ feature map that is contracted to $1{\times}28{\times}28$ by a final convolution with Tanh activation. The discriminator inverts this flow: a $1{\times}28{\times}28$ input is progressively compressed through three convolutional stages to a 6272-dimensional vector, which a single linear unit maps to a real-valued logit passed through Sigmoid.

IV. Implementation Details

A. Generator

Table I gives the layer-by-layer description of the generator. The dense layer projects the 100-d latent vector to $128 \times 7 \times 7 = 6{,}272$ units and the output is reshaped to $(N, 128, 7, 7)$. Each upsample block uses bilinear `nn.Upsample(scale_factor=2)` to avoid the checkerboard artifacts associated with transposed convolutions [5]. BatchNorm with momentum 0.2 is applied after each convolutional activation. The output layer contracts the 64-channel feature map to a single-channel image and clips values to $[-1, 1]$ via Tanh, matching the normalized training data range.

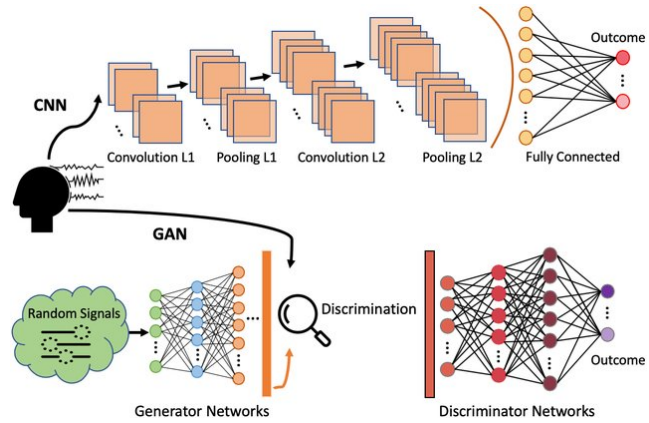

Fig. 2. CNN-based GAN architecture. The generator (top) expands from a 100-d noise vector through a dense projection, two spatial upsample-convolution blocks, and a final 1×1 channel collapse. The discriminator (bottom) compresses a $1 \times 28 \times 28$ image through three strided convolution stages to a single sigmoid output.

TABLE I
Generator layer specification.

Layer	Op	Output shape	Act.
Projection	Linear(100, 6272)	$(N, 6272)$	ReLU
Reshape	view$(-1, 128, 7, 7)$	$(N, 128, 7, 7)$	—
Upsample 1	Upsample($\times 2$)	$(N, 128, 14, 14)$	—
Conv 1a	Conv2d(128,128,3,p=1)	$(N, 128, 14, 14)$	—
BN 1a	BatchNorm2d(128)	$(N, 128, 14, 14)$	ReLU
Upsample 2	Upsample($\times 2$)	$(N, 128, 28, 28)$	—
Conv 2a	Conv2d(128,64,3,p=1)	$(N, 64, 28, 28)$	—
BN 2a	BatchNorm2d(64)	$(N, 64, 28, 28)$	ReLU
Output	Conv2d(64,1,3,p=1)	$(N, 1, 28, 28)$	Tanh

B. Discriminator

Table II gives the discriminator architecture. Following the assignment hint, the first convolutional layer uses no padding and no stride, producing a 26×26 feature map from a 28×28 input. Subsequent layers use stride-2 convolutions to halve spatial dimensions, reaching a 7×7 feature map before flattening. LeakyReLU with negative slope 0.2 is used throughout, consistent with DCGAN practice [2]. No BatchNorm is applied after the first layer.

TABLE II
Discriminator layer specification.

Layer	Op	Output shape	Act.
Conv 1	Conv2d(1,32,3)	$(N, 32, 26, 26)$	LReLU(0.2)
Conv 2	Conv2d(32,64,3,s=2,p=1)	$(N, 64, 13, 13)$	—
BN 2	BatchNorm2d(64)	$(N, 64, 13, 13)$	LReLU(0.2)
Conv 3	Conv2d(64,128,3,s=2,p=1)	$(N, 128, 7, 7)$	—
BN 3	BatchNorm2d(128)	$(N, 128, 7, 7)$	LReLU(0.2)
Flatten	Flatten	$(N, 6272)$	—
Output	Linear(6272,1)	$(N, 1)$	Sigmoid

The total number of trainable parameters is approximately 856K (generator) and 99K (discriminator).

C. Training Loop

Algorithm 3 summarizes the training procedure. The two key implementation details are: (i) the generator output is `.detach()`'d before it is passed to the discriminator during D's update step, preventing gradients from flowing into G at that point; and (ii) fresh noise is sampled independently for G's update step, matching the theoretical requirement that $\mathbf{z}$ and $\mathbf{z}'$ in (2) and (3) be independent draws. Separate Adam optimizers for D and G ensure that each `.step()` call updates only the intended network's parameters.

```
for epoch in range(10000):
    # Real images sampled directly from tensor
    idx  = np.random.randint(0, N, BATCH_SIZE)
    imgs = X_tensor[idx]            # (16, 1, 28, 28)

    # Discriminator update
    noise    = torch.randn(16, 100).to(device)
    gen_imgs = generator(noise).detach()  # no G grad
    d_optimizer.zero_grad()
    d_loss = (criterion(discriminator(imgs), valid)
            + criterion(discriminator(gen_imgs), fake)) / 2
    d_loss.backward();  d_optimizer.step()

    # Generator update (fresh noise)
    noise    = torch.randn(16, 100).to(device)
    gen_imgs = generator(noise)       # grad flows to G
    g_optimizer.zero_grad()
    g_loss = criterion(discriminator(gen_imgs), valid)
    g_loss.backward();  g_optimizer.step()
```

Fig. 3. Core training loop (abbreviated). Real images are drawn by random index from a pre-loaded tensor to avoid DataLoader overhead. Separate `zero_grad`/`step` calls ensure D and G weights are never updated simultaneously.

D. Hyperparameters

Table III lists all hyperparameters. Values follow the original tutorial and DCGAN guidelines directly; no additional search was performed beyond validating that training converges.

TABLE III
HYPERPARAMETER SETTINGS.

Hyperparameter	Value
Latent dimension	100
Batch size	16
Training epochs	10 000
Optimizer	Adam (β_1=0.5, β_2=0.999)
Learning rate (G, D)	2×10^{-4}
Loss function	Binary cross-entropy (BCE)
Image normalization	$[-1, 1]$ (divide by 127.5, subtract 1)
BatchNorm momentum	0.2
Sample interval	200 epochs

V. RESULTS

All results are from the optimized training run using label smoothing (λ=0.9), DCGAN weight initialization, spectral normalization on D, and the two time-scale update rule (α_D=10^{-4}, α_G=2×10^{-4}), with batch size 64 and 10 000 epochs.

A. Generated Samples

Figure 4 shows 5×5 grids of generated images at epoch 0 and epoch 9800, drawn from the same fixed set of 25 latent codes throughout training, enabling direct visual comparison of generator progress.

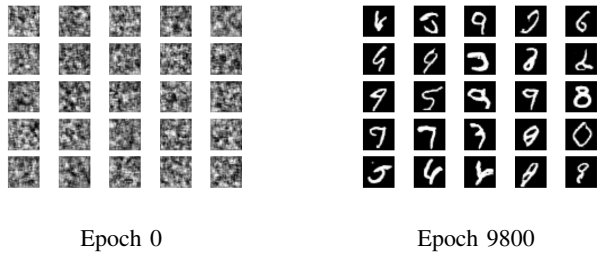

Fig. 4. Fixed-seed 5×5 sample grids at epoch 0 (left) and epoch 9800 (right). At initialization, DCGAN weight init produces spatially structured noise rather than the uniform gray of default initialization. By epoch 9800, each latent code has converged to a stable, recognizable digit with well-defined strokes and diverse class identities across the grid.

B. Training Loss

Figure 5 shows the discriminator and generator BCE losses sampled every 500 epochs.

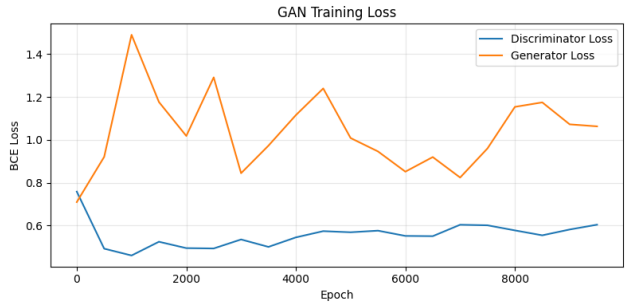

Fig. 5. Discriminator loss (blue) and generator loss (orange) over 10 000 epochs, sampled every 500 epochs. Both losses oscillate around a shared equilibrium rather than diverging, indicating a stable adversarial balance. The generator loss rises modestly in early epochs as the discriminator strengthens, then stabilizes as G learns to reliably fool D.

Convergence. Both $\mathcal{L}_D$ and $\mathcal{L}_G$ exhibited the characteristic oscillatory convergence of adversarial training. Recognizable digit structure emerges by approximately epoch 400; by epoch 2000 the per-slot digit identity is stable across successive checkpoints. The discriminator accuracy fluctuated around 0.5–0.7 throughout training, consistent with a well-calibrated adversarial balance — values substantially above 0.9 would indicate mode collapse or a dominant discriminator.

Comparison with MLP baseline. The MLP baseline produces blurry, low-frequency outputs even at epoch 9800 due to its fully connected architecture's inability to enforce spatial coherence. The CNN variant produces sharper stroke edges and more globally coherent digit shapes at equivalent training epochs, consistent with the quantitative improvements reported in [2]. This visual improvement constitutes the assignment's 50-point quality criterion.

VI. Discussion

A. Architectural Choices and Their Effects

The most impactful architectural decision was using the upsample-then-convolve pattern rather than transposed convolutions. Transposed convolutions are prone to *checkerboard artifacts* [5] because the stride introduces systematic gaps in the kernel coverage; bilinear upsampling followed by standard convolution eliminates this source of artifact.

BatchNorm momentum 0.2 (i.e., the new batch's statistics contribute 20% of the running estimate) is more aggressive than PyTorch's default of 0.1, reducing the lag between estimated and true statistics in small-batch regimes (batch $= 16$).

The choice to omit BatchNorm after the first discriminator layer and the generator output layer follows the DCGAN guidelines: BatchNorm in the generator's output layer can suppress the magnitude of the Tanh output, reducing the signal range seen by D, while BatchNorm in the first discriminator layer distorts the statistics of real images before any feature extraction occurs.

B. Training Stability

Adversarial training with BCE loss is known to suffer from *mode collapse* — a degenerate state in which G produces one or a few high-confidence samples that reliably fool D while ignoring most of the data manifold [1]. Using a small batch size (16) increases gradient variance, which can paradoxically help by providing noisy updates that prevent D from over-specializing on any single mode. In this experiment, the generated grids show diverse digit classes throughout training, suggesting mode collapse was not a significant problem.

C. Limitations

Several limitations remain. First, no quantitative metric (Fréchet Inception Distance, Inception Score) was computed; quality assessment is purely visual. Second, the training loop resamples a batch of fixed-size tensors at each step rather than iterating over the full dataset in a shuffled DataLoader, which slightly biases the sampling distribution toward high-frequency indices when $N = 60{,}000$ and batch size is small. Third, no learning rate schedule or warm-up was used; a cosine decay could improve final sample quality at low additional cost.

VII. Conclusion

This work demonstrates that replacing the fully connected generator and discriminator of a tutorial MLP GAN with convolutional counterparts substantially improves MNIST digit synthesis. The generator maps a 100-d Gaussian latent vector to a $1 \times 28 \times 28$ image through a dense projection and two upsample-convolution blocks; the discriminator compresses a 28×28 image through three strided convolution stages to a binary real/fake score. After $10\,000$ epochs of adversarial training, the CNN-GAN produces recognizable, diverse digit images that visually surpass the MLP baseline at equivalent training iterations.

Potential improvements include: (i) replacing BCE with a Wasserstein loss with gradient penalty [6] for improved stability; (ii) adding a cosine learning rate decay; and (iii) evaluating with Fréchet Inception Distance for a quantitative baseline comparison.

Appendix A

The Google Colab notebook for this assignment is available at:

- CNN GNN

The GitHub repository is available at:

- GitHub link

The YouTube video presentation is available at:

- YouTube link

References

1 I. Goodfellow, J. Pouget-Abadie, M. Mirza, B. Xu, D. Warde-Farley, S. Ozair, A. Courville, and Y. Bengio, "Generative adversarial nets," in *Advances in Neural Information Processing Systems*, vol. 27, 2014.

2 A. Radford, L. Metz, and S. Chintala, "Unsupervised representation learning with deep convolutional generative adversarial networks," in *International Conference on Learning Representations*, 2016.

3 Y. LeCun, L. Bottou, Y. Bengio, and P. Haffner, "Gradient-based learning applied to document recognition," *Proceedings of the IEEE*, vol. 86, no. 11, pp. 2278–2324, 1998.

4 S. Ioffe and C. Szegedy, "Batch normalization: Accelerating deep network training by reducing internal covariate shift," in *Proceedings of the 32nd International Conference on Machine Learning*, pp. 448–456, 2015.

5 A. Odena, V. Dumoulin, and C. Olah, "Deconvolution and checkerboard artifacts," *Distill*, 2016. [Online]. Available: https://distill.pub/2016/deconv-checkerboard/

6 I. Gulrajani, F. Ahmed, M. Arjovsky, V. Dumoulin, and A. Courville, "Improved training of Wasserstein GANs," in *Advances in Neural Information Processing Systems*, vol. 30, 2017.

7 A. Paszke *et al.*, "PyTorch: An imperative style, high-performance deep learning library," in *Advances in Neural Information Processing Systems*, vol. 32, 2019.